

Proyecto 03: Ensayos Clínicos.

Este documento contiene las especificaciones de funcionalidad y requerimientos para el **tercer** proyecto de programación de la asignatura **Estructura de Datos y de la Información** en el curso 2022/23.

1. Introducción.

Nuestro objetivo será desarrollar una aplicación que realice la gestión básica de un **ensayo clínico sobre los pacientes del hospital** que venimos desarrollando en las entregas anteriores. La aplicación tendrá una funcionalidad muy limitada, pero debe cumplir con las siguientes características generales:

- **Eficiencia:** debido a la gran cantidad de datos, el programador debe encontrar la forma de almacenar y procesar los datos de la forma más eficiente posible. Además, debe realizar un análisis de la complejidad de la aplicación para cumplir este requisito. En esta entrega se pondrá especial atención a esta característica del código desarrollado.
- **Extensibilidad:** el diseño de la aplicación en cuanto a entidades y estructuras de datos debe ser extensible, para facilitar el desarrollo de futuras versiones de la aplicación y ampliar su funcionalidad. Sin embargo, la interfaz (métodos públicos) de la clase principal está preestablecida, y debe utilizarse sin cambios de nombre ni parámetros.
- **Modularidad:** la organización de las entidades que conforman la aplicación debe ser modular, de forma que las modificaciones y ampliaciones de la misma se puedan realizar de la forma más sencilla posible, afectando a la menor cantidad de código.

Para cumplir con los requisitos anteriores se seguirá una metodología de desarrollo basada en el paradigma de *Programación Orientada a Objetos* y se utilizará el lenguaje de programación **C++**, concretamente la versión (dialecto) 11. **El alumno debe asegurarse de que su código compila bajo esta versión del lenguaje y no incorpora características propias de ningún sistema operativo.**

El **funcionamiento general básico** de la aplicación es secuencial (no basada en menú), y comprende los siguientes pasos:

1. Crear el ensayo clínico que contendrá, entre otras cosas, el **nombre del alumno**.
2. Los datos de los pacientes incluidos en el fichero `pacientes_ordenados.csv` se importarán en una estructura de datos de tipo árbol binario de búsqueda de pacientes.
3. Se lee un segundo fichero (`ensayo.csv`) con las puntuaciones que habrá que añadir a cada paciente.
4. Se muestran por pantalla los N pacientes con mayor puntuación (siendo N un parámetro de la operación que implementa esta funcionalidad).
5. Como el fichero `ensayo.csv` puede contener errores derivados de la toma de muestras en el ensayo, los errores se almacenarán (sin duplicados) y podrán ser mostrados con posterioridad.
6. Finalmente, se mostrarán algunos datos del proceso, como los niveles del árbol o algunos de los pacientes por apellido o DNI.

Los profesores proporcionarán un esqueleto de código que **debe funcionar sin modificaciones**. La eficiencia de vuestros algoritmos será muy importante, ya que, a los 20 alumnos que

consigan ejecutar los pasos del 1 al 4 de la lista anterior con mayor rapidez (siempre que cumplan el resto de los requisitos) **se les otorgará un punto extra en el proyecto**. Se añaden utilidades al esqueleto del proyecto para que el desarrollador pueda **medir el tiempo de ejecución**.

El resto del documento se organiza de la siguiente forma. En la **sección 2**, se especifica el código a partir del cual se puede comenzar la implementación del proyecto. En la **sección 3**, se enumeran los algoritmos básicos que debemos implementar y, en la **sección 4**, se discuten los mecanismos de análisis y tests que debemos aplicar sobre nuestra aplicación para evaluarla y asegurar que cumple los requisitos de la forma más eficiente posible. Finalmente, en la **sección 5**, **se establecen los criterios de evaluación y calificación del proyecto**, así como las fechas de entrega.

2. Creación del código base para el proyecto.

El proyecto contendrá algunas de las clases implementadas durante las sesiones de laboratorio. El profesorado, además, proporciona un esqueleto (disponible en el aula virtual) que el programador debe utilizar, **sin modificar**, como base para el desarrollo de su solución:

1. **Paciente**: Clases base que contiene la información de un paciente del hospital.
2. La clase `EnsayoClinico`. Es la clase principal del proyecto e implementa su funcionalidad. El nombre de la clase y su interfaz pública deben utilizarse sin modificación, esto es, deben respetarse el nombre y los parámetros de todos los métodos públicos. Esta clase contendrá, al menos, dos atributos: el *nombre* del ensayo (que debe ser el nombre y apellidos del alumno) y el *árbol de pacientes*.
3. Los ficheros **.csv** con los datos que hay que importar (`pacientes_ordenados.csv` y `ensayo.csv`).
4. Un programa principal (`prog_principal.cpp`) que implementará el tratamiento de los datos de un ensayo clínico. El programa principal será proporcionado por los profesores de la asignatura y **debe poder ejecutarse correctamente sin cambios**. El alumno puede implementar extensiones para realizar las pruebas de ejecución necesarias.
5. Una clase que permite medir el tiempo transcurrido entre dos instantes y que permite al usuario evaluar la eficiencia de sus algoritmos (`Timer`).

A partir de la estructura de código anterior, se implementará la funcionalidad solicitada en forma de algoritmos en la clase `EnsayoClinico`, que se ejecutarán desde el programa principal. Se pueden añadir otras clases al proyecto si se considera necesario.

Implementación de las clases base.

La clase `Paciente` se puede reutilizar de las fases anteriores, si bien deberá extenderse con, al menos, un nuevo atributo de tipo entero (y los *setters/getters* correspondientes) para anotar la puntuación de cada paciente en el ensayo clínico.

Datos.

Los datos a importar han sido *pretratados* y están a disposición del programador en el Aula Virtual de la asignatura en forma de ficheros `.csv`. Se deben utilizar los datos proporcionados en los ficheros **sin modificaciones**. Cualquier modificación supondrá que los resultados de los algoritmos no serán los esperados y, como consecuencia, se considerarán incorrectos.

Los archivos que contienen los datos son de tipo texto y se enumeran en la tabla siguiente:

<code>pacientes_ordenados.csv</code>	Fichero que contiene los pacientes del hospital. La información de cada paciente será: <i>DNI, Nombre, apellidos, género</i> (0: Hombre, 1: Mujer, 2: NoDefinido) y <i>edad</i> . La información en el fichero está ordenada por DNI.
<code>ensayo.csv</code>	Fichero que contiene las anotaciones sobre el ensayo clínico. La información viene en forma de: <i>DNI-paciente, puntuación</i> . La puntuación se añadirá al paciente.

Contenedores de datos.

La estructura de datos básica que vamos a utilizar en esta fase del proyecto es un **árbol binario de búsqueda** en el que se almacenarán los pacientes. Este es un requisito del proyecto. Para el resto de datos, el alumno debe decidir qué estructura utilizar de las que hemos visto en la asignatura (incluidas vector, pila, cola, lista y árbol), siempre pensando en obtener la mayor eficiencia posible.

Ejemplo de salida por pantalla.

El siguiente es un ejemplo de la salida que se obtiene en la ejecución del programa principal proporcionado (`prog_principal.cpp`). El alumno debe realizar más pruebas de ejecución y las pruebas unitarias como se especifica en la sección 4.

```

Ensayo de: NOMBRE y APELLIDOS del alumno

Mostrar 10 pacientes con mayor puntuación:
Yahya El Baroudi El Ouazghari: 37761577D [Mujer] *5265*
Jaime Quijada González: 21690145W [Hombre] *5232*
Daniel Becerra Benítez: 96761557J [NoDefinido] *5230*
Carlos Arroyo Caballero: 35606489L [NoDefinido] *5226*
Enrique Márquez García: 10224306E [NoDefinido] *5222*
Pablo Márquez Hernández: 72046317O [Hombre] *5221*
Daniel Hernández Marín: 14429687G [Hombre] *5213*
Iván López García: 47195949T [NoDefinido] *5213*
Antonio Villalba Rapela: 30050047V [Mujer] *5208*
David Santos Pallarés: 48323930Q [Mujer] *5206*

Errores en pacientes:
17281002S
11324230Z
99217882B
99812892J
73092301G

Tiempo: 3.3151

Niveles del arbol: 8

```

Pacientes cuyo apellido contiene la cadena -art-:

```
Francisco Javier Martín Ramírez: 30001766J [Hombre] *5020*
Máximo Luis Bueno Martínez: 43937451Z [Hombre] *5131*
Daniel Sánchez Martín: 50490564M [NoDefinido] *5020*
Arturo Martín Bueno: 55906031V [NoDefinido] *5047*
Jorge Méndez Martínez: 63670540B [NoDefinido] *4946*
Ismael Miguel Martín: 68711824N [Mujer] *4909*
Julio Martín De La Flor: 91942490Q [NoDefinido] *4904*
Álvaro Mendo Martín: 92125171D [NoDefinido] *5030*
Hugo Martín Gabriel: 97773647B [Hombre] *4843*
```

Pacientes cuyo DNI empieza por la cadena -28-:

```
Adrián Caballero Ramos: 28436484O [NoDefinido] *4921*
Alejandro Nicolas Rodriguez Galan: 28659346N [Mujer] *5078*
David Muñoz San Andrés: 28661130E [NoDefinido] *5108*
Antonio Manuel González Arce: 28925342J [NoDefinido] *5082*
```

3. Algoritmos a implementar.

La funcionalidad de los ensayos clínicos a implementar es muy limitada, pero tiene unos requerimientos muy estrictos en cuanto a interfaz y eficiencia de los algoritmos. De esta forma, se deben seguir los siguientes puntos:

1. Los pacientes del fichero proporcionado deben ser cargados en un árbol binario de búsqueda, de forma que las búsquedas de pacientes por DNI sean lo más eficientes posible. Por tanto, el árbol debe estar lo más **equilibrado** posible.
2. Los datos del ensayo proporcionados en el fichero deben ser leídos y anotados para cada paciente. Esto quiere decir que, por cada línea del fichero, sumaremos los puntos al paciente concreto. **Algunos de los datos del fichero pueden contener errores** (que el DNI no se corresponda con ningún paciente del árbol) que se deben controlar convenientemente.
3. Además de esta funcionalidad, se incluyen algunos algoritmos auxiliares que se deben implementar y que se describen a continuación.

Los algoritmos a implementar en nuestra aplicación son los siguientes:

1. Mostrar los N pacientes con mayor puntuación en el ensayo. El formato para mostrar los datos de los pacientes debe ser similar a (uno por línea):

```
DNI-paciente # Nombre y apellidos # edad # género # puntuación.
```

2. Mostrar todos los errores al anotar la puntuación del ensayo. **No se repetirán errores**, es decir, que si el DNI X es erróneo, sólo se mostrará el DNI X una vez, con el formato:

```
DNI-erróneo-1
DNI-erróneo-2
. . .
```

3. Mostrar el número de niveles del árbol construido a partir de los datos de pacientes.
4. Mostrar todos los datos de los pacientes cuyo **apellido contenga** una subcadena proporcionada como parámetro.

5. Mostrar todos los datos de los pacientes cuyo **DNI empiece** por una subcadena proporcionada como parámetro.

Todos los algoritmos deben ser implementados sobre las estructuras de datos en memoria, en las que, previamente, se habrá cargado la información desde fichero, es decir, los algoritmos anteriores no podrán implementarse directamente sobre los datos del fichero.

4. Análisis, pruebas y documentación.

Además del código que implementa la funcionalidad descrita en las secciones anteriores, para obtener la calificación de aprobado será obligatorio incluir:

- Juego de pruebas unitarias para la clase `EnsayoClinico`. No será necesario incluir prueba para ninguna otra clase implementada.
- La documentación interna correspondiente a cada operación de las clases implementadas. Esta documentación incluirá **precondiciones, descripción y complejidad**. Se documentarán también los aspectos que el alumno considere oportunos sobre los detalles de su código.

5. Evaluación.

La rúbrica con la que se evaluará el proyecto está disponible en la actividad de entrega. Tal y como se indica en ella, los requisitos mínimos que deben cumplir los proyectos entregados para ser evaluados, son los siguientes:

- Tal y como se indica en la ficha de la asignatura, *la copia o el plagio en cualquier actividad, proyecto o prueba, ya sea en una parte o en su totalidad, supone una nota final de SUSPENSO (0) en la convocatoria y una nota de 0 en todas las calificaciones obtenidas hasta el momento para todas las personas implicadas, además de las actuaciones legales pertinentes.*
- Aquellos proyectos que no compilen o que no funcionen correctamente en la mayoría de los casos, no se corregirán y serán calificados con una nota de 0.
- Del mismo modo, aquellos proyectos que no incluyan la documentación interna de todas las clases o pruebas significativas para todas ellas, tampoco serán corregidos y serán calificados con una nota de 0.
- Para la implementación del proyecto solo podrá utilizarse C++, en su versión (dialecto) C++11. No está permitido utilizar la biblioteca STL (*Standard Template Library*) en el desarrollo, ni cualquier otra biblioteca similar. No está permitido incluir código dependiente del sistema operativo.

Los proyectos entregados deberán cumplir las siguientes normas:

- Deberán realizarse **de forma individual**.
- Deberán entregarse en un fichero `zip` cuyo nombre seguirá el formato `ApellidosAlumno.zip` y cuyo contenido incluirá los archivos fuente (`.h` y `.cpp`) del código y pruebas desarrolladas, con su documentación interna.

Aquellos proyectos que cumplan los requisitos y normas anteriores, se evaluarán teniendo en cuenta los siguientes criterios:

- Uso de **punteros** en estructuras de datos, evitando así los problemas derivados de mover datos en memoria (*eficiencia*) y mantener distintas copias (*consistencia*).
- Diseño correcto de interfaces, incluidas la documentación (*precondiciones y descripción*) y evaluación de la *complejidad* de las operaciones.

- Organización correcta de los datos en memoria de forma que la implementación de los algoritmos cumpla con los requisitos de eficiencia y uso de memoria.
- Corrección en el resultado de la ejecución de los algoritmos.
- Uso correcto y eficiente de parámetros en la invocación de operaciones.
- Uso de constructores y destructores para las clases de forma coherente y adecuada, incluido el constructor por copia.
- Siempre que sea posible, no se duplicarán objetos en la aplicación, y se evitarán copias y duplicados de los mismos.

La fecha límite de entrega del proyecto es el 12 de mayo a las 23:55 h, a través de la correspondiente tarea abierta en el Aula Virtual de la asignatura. No se tendrán en cuenta entregas realizadas por cualquier otro medio que no sea el establecido, ni con posterioridad a esa fecha.